

FGV EMap

João Pedro Jerônimo e Eduardo Adame

Aprendizado de Máquina

Revisão para A3

Rio de Janeiro

2026

Conteúdo

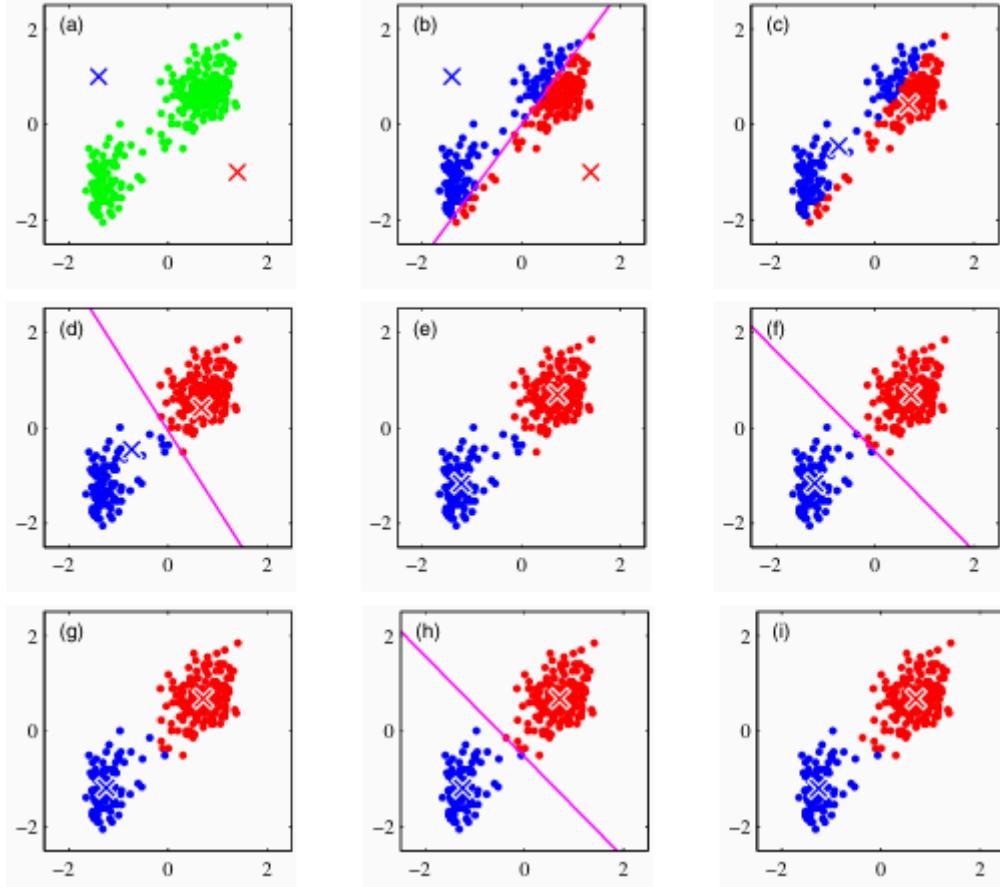
1	K-means	3
1.1	Introdução	4
1.2	O Algoritmo	4
2	Principal Component Analysis	7
2.1	Introdução	8
2.2	Definições	8
2.3	Máxima Variância	9
2.4	Minimizando o Erro de Projeção	9
3	Gaussian and Bernoulli Mixture Models	11
3.1	Introdução e Definição	12
3.2	Máxima Verossimilhança	14
3.3	Expectation-Maximization (EM) para GMMs	14
3.4	Algoritmo EM Variacional	17
3.5	Bernoulli Mixture Models	18
3.6	Singularidades e Identificabilidade	22
4	Variational Autoencoders	24
4.1	Introdução	25
4.2	Autoencoders Determinísticos	25
4.2.1	Autoencoders Profundos	25
4.2.2	Autoencoders Esparsos	26
4.2.3	Denoising Autoencoders	26
4.3	Autoencoders Variacionais	27
4.3.1	Inferência Amortizada	28
4.3.2	Truque da Reparametrização	29
5	Generative Adversarial Networks	31
5.1	Introdução	32
5.2	Treinamento Adversarial	32
5.3	Função de Custo	32
5.4	Treinamento do GAN	33
	Referências	34

K-means

1.1 Introdução

A partir desse momento, vamos começar a estudar métodos de machine learning não supervisionados. Diferente do aprendizado supervisionado, onde temos um conjunto de dados rotulado, no aprendizado não supervisionado, os dados não possuem rótulos e o objetivo é encontrar padrões ou estruturas subjacentes nos dados.

O primeiro que vamos estudar é um dos algoritmos de machine learning não supervisionado mais simples que existe, o K-means. Ele é um algoritmo de clustering que busca particionar os dados em K grupos distintos com base em suas características.



1.2 O Algoritmo

Considere o problema de classificar pontos em um espaço. Naturalmente, pensamos que os pontos mais próximos um do outro devem pertencer ao mesmo grupo. O K-means é um algoritmo que busca encontrar esses grupos de forma iterativa, ajustando os centróides dos clusters até que a convergência seja alcançada.

Suponha que temos um dataset $D = \{x_1, \dots, x_N\}$ com pontos aleatórios de um espaço euclidiano de dimensão D . O nosso objetivo é particionar o dataset em um conjunto de K clusters, onde, no momento, vamos supor que K é conhecido. Também definimos um conjunto de K centróides $\mu_1, \dots, \mu_K$. Nosso objetivo então é associar cada um dos pontos x_i a um dos centróides μ_k , de forma que a soma das distâncias quadradas entre os pontos e seus centróides seja minimizada. Definimos também uma variável de associação $r_{nk} \in \{0, 1\}$, que indica se o ponto x_n pertence ao cluster k ou não. Então podemos definir nossa função de custo, também conhecida como *função de distorção*, como:

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|x_n - \mu_k\|^2 \quad (1)$$

que é a soma das distâncias quadradas entre cada ponto e o centróide do cluster ao qual ele pertence. O objetivo do K-means é minimizar essa função de custo.

Teorema 1.2.1 (Valor ótimo de r_{nk}): Para um conjunto fixo de centróides $\mu_1, \dots, \mu_K$, a escolha ótima de r_{nk} é dada por:

$$r_{nk} = \begin{cases} 1 & \text{se } k = \operatorname{argmin}_j \|x_n - \mu_j\|^2 \\ 0 & \text{caso contrário} \end{cases} \quad (2)$$

Demonstração: Perceba que (1) é uma função linear de r_{nk} . Os termos envolvendo diferentes n são independentes entre si, então podemos otimizar para cada n separadamente de forma que $r_{nk} = 1$ para qualquer valor k que minimize $\|x_n - \mu_k\|^2$ e $r_{nk} = 0$ para todos os outros valores de k . Ou seja, simplesmente assinalamos cada ponto ao cluster cujo centróide está mais próximo (o que pode ser expresso como $k = \operatorname{argmin}_j \|x_n - \mu_j\|^2$). $\square$

Teorema 1.2.2 (Valor ótimo de μ_k): Para um conjunto fixo de associações r_{nk} , a escolha ótima de μ_k é dada por:

$$\mu_k = \frac{\sum_{n=1}^N r_{nk} x_n}{\sum_{n=1}^N r_{nk}} \quad (3)$$

Demonstração: Perceba que (1) é uma função quadrática de μ_k . Para encontrar o valor ótimo de μ_k , podemos derivar a função de custo em relação a μ_k e igualar a zero:

$$\frac{\partial J}{\partial \mu_k} = 2 \sum_{n=1}^N r_{nk} (\mu_k - x_n) = 0 \quad (4)$$

Rearranjando os termos, obtemos:

$$\mu_k \sum_{n=1}^N r_{nk} = \sum_{n=1}^N r_{nk} x_n \quad (5)$$

Dividindo ambos os lados por $\sum_{n=1}^N r_{nk}$, obtemos a expressão desejada para μ_k . $\square$

Perceba que no Teorema 1.2.2, o centróide μ_k é simplesmente a média de todos os pontos que pertencem ao cluster k . Isso faz sentido intuitivamente, pois queremos que o centróide represente o “centro” do cluster.

Perceba, porém, que o (1) não é uma função convexa de r_{nk} e μ_k juntos, então não podemos otimizar ambos ao mesmo tempo. O K-means resolve isso alternando entre otimizar r_{nk} e μ_k iterativamente até que a convergência seja alcançada.

Além disso, o K-means é sensível à inicialização dos centróides. Diferentes inicializações podem levar a diferentes soluções locais, então é comum executar o algoritmo várias vezes com diferentes inicializações e escolher a melhor solução encontrada.

Note que eu mostrei uma forma de aplicar o K-means em batch, ou seja, considerando todos os pontos de uma vez. Existe também uma versão estocástica online do K-means, onde os centróides

são atualizados à medida que novos pontos chegam. Não entraremos em detalhes da derivação, mas esse método se utiliza do procedimento de Robbins-Monro para atualizar os centróides de forma incremental. Esse procedimento é utilizado para resolver equações do tipo

$$\mathbb{E}[Z(\theta)] = 0 \quad (6)$$

onde não observamos $Z(\theta)$ diretamente, mas sim uma amostra $Z_{n(\theta)}$ ruidosa de $Z(\theta)$. Em Machine Learning Pattern and Recognition[1], Bishop nota que a equação do gradiente de ∇J é exatamente no formato de (6)

$$\sum_{n=1}^N r_{nk}(\mu_k - x_n) = 0 \quad (7)$$

então chegamos no procedimento de atualização estocástico do K-means, que é dado por:

$$\mu_k^{t+1} = \mu_k^t + \eta_n(x_n - \mu_k^t) \quad (8)$$

onde η_n é a taxa de aprendizado, que deve ser escolhida de forma que ela diminua quanto mais pontos forem vistos, para garantir a convergência do algoritmo.

Algoritmo K-means (Batch)

```

1 function K-means(D, K) {
2   initialize  $\mu_1, \dots, \mu_K$  randomly
3   repeat {
4     for  $n = 1$  to  $N$  {
5        $r_{nk} = 0$  for all  $k$ 
6        $r_{nk} = 1$  for  $k = \operatorname{argmin}_j \|x_n - \mu_j\|^2$ 
7     }
8     for  $k = 1$  to  $K$  {
9        $\mu_k = \frac{\sum_{n=1}^N r_{nk} x_n}{\sum_{n=1}^N r_{nk}}$ 
10    }
11  }
12  until convergence
13 }
```

O algoritmo do K-means é baseado na distância euclidiana para medir a discrepância entre os pontos e os centróides. No entanto, essa medida não é adequada em todos os cenários. Por exemplo, se os clusters forem labels? Além de que torna J não resistente à outliers. Para lidar com isso, podemos utilizar o *K-medoids*, que é uma variação do K-means que utiliza uma outra distorção qualquer $\mathcal{V}(x, x')$ em vez da média para calcular os centróides.

$$\tilde{J} = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \mathcal{V}(x_n, \mu_k) \quad (9)$$

Principal Component Analysis

2.1 Introdução

Nós que trabalhamos com análise de dados, muitas vezes nos deparamos com datasets assustadores, com muitas dimensões, o que acaba dificultando nossa capacidade de fazer análises e visualizações. Visando solucionar esse nosso problema, é que o PCA entra em cena. Ele é um método de redução de dimensionalidade que busca encontrar uma representação mais compacta dos dados, preservando ao máximo a variância dos dados originais. O PCA é amplamente utilizado em diversas áreas, como reconhecimento de padrões, compressão de imagens e análise exploratória de dados.

2.2 Definições

O PCA pode ser interpretado como duas definições distintas que dão origem ao mesmo resultado

- Projeção ortogonal dos dados em um espaço de menor dimensão (Subespaço Principal) de forma que a variância dos dados seja maximizada
- Projeção Linear que minimiza o custo médio de projeção

Antes de entrarmos em detalhes sobre essas definições, vamos colocar algumas definições úteis que, nós já sabemos, mas refrescar a nossa memória nunca é demais

Definição 2.2.1 (Projetar sobre um subespaço): Dado um ponto $x \in \mathbb{R}^D$, sua projeção ortogonal no subespaço é o ponto $\hat{x}$ nesse subespaço mais próximo de x . Como consequência, temos que

$$(x - \hat{x})^T u = 0 \quad (10)$$

onde u é um vetor qualquer no subespaço

Teorema 2.2.1 (Projeção ortogonal): Seja X uma matriz $\mathbb{R} \times \mathbb{R}$, a projeção do vetor y no espaço coluna de X é dada por:

$$\hat{y} = X(X^T X)^{-1} X^T y \quad (11)$$

se X é ortogonal, então

$$\hat{y} = X X^T y \quad (12)$$

Teorema 2.2.2 (Coeficientes de Base): Seja $\{q_1, \dots, q_D\}$ uma base ortogonal do $\mathbb{R}^D$ e $x \in \mathbb{R}^D$ tal que

$$x = \sum_{i=1}^D \alpha_i q_i \quad (13)$$

então temos que

$$Qx = (\alpha_1 \dots \alpha_D)^T \quad (14)$$

onde Q é a matriz cujas colunas são os vetores da base.

2.3 Máxima Variância

Seja $X \in \mathbb{R}^{N \times D}$ onde x_i^T é a i -ésima linha de X , queremos projetar X em um subespaço $\mathbb{R}^M$ com $M < D$ enquanto maximizamos a variância dos dados projetados.

Supondo que $M = 1$, pegamos $u_1 \in \mathbb{R}^D$ tal que $u_1^T u_1 = 1$ então pegamos quanto de u_1 compõe o vetor x_i com $u_1^T x_i$ (Teorema 2.2.2). Vamos definir a média dos dados projetados como

$$u_1^T \bar{x} = \frac{1}{N} \sum_{i=1}^N u_1^T x_i \quad (15)$$

e também definimos a variância deles como

$$u_1^T S u_1 = \frac{1}{N} \sum_{i=1}^N (u_1^T x_i - u_1^T \bar{x})^2 \quad (16)$$

e

$$S = \frac{1}{N} \sum_{n=1}^N (x_n - \bar{x})(x_n - \bar{x})^T \quad (17)$$

Agora, queremos maximizar $u_1^T S u_1$ com respeito a u_1 com a restrição de $u_1^T u_1 = 1$. Antes de fazermos isso mesmo, a lógica desse processo é que queremos entender qual a direção do espaço que mais contribui com a variância dos dados, ou seja, qual a direção que mais “espalha” os dados. Para isso, vamos utilizar o método de multiplicadores de Lagrange para maximizar $u_1^T S u_1$ com a restrição de $u_1^T u_1 = 1$. Definimos a função lagrangiana como:

$$\mathcal{L}(u_1, \lambda) = u_1^T S u_1 - \lambda(u_1^T u_1 - 1) \quad (18)$$

Realizando as contas necessárias, chegamos que

$$S u_1 = \lambda u_1 \quad (19)$$

Ou seja, a direção que mais contribui com a variância dos dados é o autovetor de S correspondente ao maior autovalor. Esse autovetor é chamado de *primeira componente principal*. Sabendo disso, podemos utilizar de **indução forte** para mostrar que a segunda componente principal é o autovetor de S correspondente ao segundo maior autovalor, e assim por diante. Dessa forma, as M primeiras componentes principais são os M autovetores de S correspondentes aos M maiores autovalores.

2.4 Minimizando o Erro de Projeção

Pegamos um set $\{u_1, u_2, \dots, u_D\}$ de vetores ortonormais em $\mathbb{R}^D$. Pelo Teorema 2.2.2, sabemos que

$$x_n = \sum_{i=1}^D (x_n^T u_i) u_i \quad (20)$$

Porém, queremos aproximar x_n usando um conjunto de só $M < D$ variáveis. Escrevemos então:

$$\hat{x}_n = \sum_{i=1}^M \underbrace{z_{ni}}_{\text{Depende de } x_n} u_i + \sum_{i=M+1}^D \underbrace{b_i}_{\text{Constante em } x_n} u_i \quad (21)$$

E queremos então minimizar

$$J = \frac{1}{N} \sum_{n=1}^N \|x_n - \hat{x}_n\|^2 \quad (22)$$

Derivando essa função de custo com relação a z_{ni} e b_i e igualando a zero e usando das condições de ortogonalidade, chegamos que

$$z_{ni} = x_n^T u_i \quad (23)$$

$$b_i = \bar{x}^T u_i \quad (24)$$

substituindo, obtemos então:

$$x_n - \hat{x}_n = \sum_{i=M+1}^D \{(x_n^T - \bar{x}^T)u_i\}u_i \quad (25)$$

com isso, conseguimos achar uma fórmula para J

$$J = \frac{1}{N} \sum_{n=1}^N \sum_{i=M+1}^D \{(x_n^T - \bar{x}^T)u_i\}^2 = \sum_{i=M+1}^D u_i^T S u_i \quad (26)$$

Agora, só nos falta otimizar J com relação à u_i e aplicar a otimização com as condições de ortogonalidade. Vamos primeiro considerar o caso $M = 1$, temos que a função de lagrange é dada por

$$\mathcal{L}(u_1, \lambda) = \sum_{i=2}^D u_i^T S u_i - \sum_{i=2}^D \lambda_i (u_i^T u_i - 1) \quad (27)$$

derivando essa função e aplicando as regras de otimização restrita, chegamos que

$$S u_i = \lambda_i u_i \quad (28)$$

Novamente, chegamos na conclusão de que os autovetores de S são as direções que minimizam o erro de projeção. E, novamente, podemos utilizar de indução forte para mostrar que os autovetores correspondentes aos maiores autovalores são as direções que minimizam o erro de projeção.

Gaussian and Bernoulli Mixture Models

3.1 Introdução e Definição

No k-means, cada ponto é atribuído a um único cluster, o que significa que a fronteira entre os clusters é rígida. No entanto, em muitos casos, pode ser mais apropriado permitir que cada ponto tenha uma probabilidade de pertencer a cada cluster. Isso nos leva aos Modelos de Mistura Gaussiana (GMMs), que é uma generalização do K-means.

Aqui, nós supomos que cada ponto **pode** ter saído de um dos K clusters, mas não sabemos de qual, cada cluster esse sendo representado por uma distribuição gaussiana. Cada cluster k é caracterizado por uma média μ_k e uma matriz de covariância Σ_k . Além disso, cada cluster tem um peso π_k , que representa a proporção de pontos que pertencem a esse cluster.

Definição 3.1.1 (Modelo de Mistura Gaussiana): Um Modelo de Mistura Gaussiana é definido como:

$$p(x) = \sum_{k=1}^K \pi_k N(x \mid \mu_k, \Sigma_k) \quad (29)$$

onde $N(x \mid \mu_k, \Sigma_k)$ é a densidade da distribuição gaussiana com média μ_k e covariância Σ_k , e π_k são os pesos dos clusters, que satisfazem $\sum_{k=1}^K \pi_k = 1$.

Teorema 3.1.1 (Validade da distribuição): Seja $p(x)$ um Modelo de Mistura Gaussiana com K componentes. Então, $p(x)$ é uma distribuição de probabilidade válida, ou seja, $p(x) \geq 0$ para todo x e $\int p(x) dx = 1$.

Demonstração: Para mostrar que $p(x) \geq 0$, note que cada termo na soma é não-negativo, pois $\pi_k \geq 0$ e $N(x \mid \mu_k, \Sigma_k) \geq 0$. Portanto, $p(x) \geq 0$ para todo x .

Para mostrar que a integral de $p(x)$ sobre todo o espaço é igual a 1, usamos a linearidade da integral:

$$\begin{aligned} \int p(x) dx &= \int \sum_{k=1}^K \pi_k N(x \mid \mu_k, \Sigma_k) dx \\ &= \sum_{k=1}^K \pi_k \int N(x \mid \mu_k, \Sigma_k) dx \\ &= \sum_{k=1}^K \pi_k * 1 \\ &= \sum_{k=1}^K \pi_k \\ &= 1 \end{aligned} \quad (30)$$

□

Vamos também introduzir o conceito de **variável latente**. Intuitivamente, uma variável latente é uma variável que não observamos diretamente, mas que influencia os dados que observamos. Por exemplo, a classe de um documento em uma análise de tópicos, já que podemos não saber que um documento fala sobre biologia, mas ele influencia nosso modelo a aprender sobre o assunto.

No caso dos GMMs, podemos introduzir uma variável latente z_n para cada ponto x_n , que indica de qual cluster o ponto foi gerado. Especificamente, z_n é um vetor one-hot de dimensão K , onde $z_{nk} = 1$ se o ponto x_n foi gerado pelo cluster k , e $z_{nj} = 0$ para $k \neq j$.

Vamos definir a distribuição conjunta de x_n e z_n como:

$$p(x_n, z_n) = p(z_n)p(x_n | z_n) \quad (31)$$

a distribuição marginal de z_n é definida em termo dos coeficientes de mistura π_k :

$$\mathbb{P}(z_{nk} = 1) = \pi_k \quad (32)$$

de forma que $\pi_k \geq 0$ e $\sum_{k=1}^K \pi_k = 1$ para que $p(z_n)$ seja uma distribuição de probabilidade válida. Por conta da forma que definimos z_n como vetor one-hot, podemos reescrever sua distribuição como:

$$p(z_n) = \prod_{k=1}^K \pi_k^{z_{nk}} \quad (33)$$

Similarmente, a distribuição condicional de x_n dado $z_{nk} = 1$ é definida como:

$$p(x_n | z_{nk} = 1) = N(x_n | \mu_k, \Sigma_k) \quad (34)$$

que também pode ser escrita na forma:

$$p(x_n | z_n) = \prod_{k=1}^K N(x_n | \mu_k, \Sigma_k)^{z_{nk}} \quad (35)$$

A distribuição conjunta é escrita então como $p(z_n)p(x_n | z_n)$ e a marginal sobre x é obtida somando sobre todas as possíveis configurações de z_n :

$$p(x_n) = \sum_{z_n} p(x_n, z_n) = \sum_{z_n} p(z_n)p(x_n | z_n) = \sum_{k=1}^K \pi_k N(x_n | \mu_k, \Sigma_k) \quad (36)$$

Pode até parecer que, representando a distribuição de x_n como uma mistura de gaussianas, estamos apenas complicando as coisas, mas a introdução da variável latente z_n nos permite trabalhar com a conjunta (que vai se mostrar ser bem mais fácil de lidar) e também nos dá uma interpretação probabilística do modelo.

Outra quantidade que será importante é a probabilidade posterior de z_n dado x_n (chamaremos de $\gamma(z_{nk})$), que é dada pelo Teorema de Bayes:

$$\begin{aligned} \gamma(z_{nk}) &= \mathbb{P}(z_{nk} = 1 | x_n) \\ &= \frac{p(z_{nk} = 1)p(x_n | z_{nk} = 1)}{p(x_n)} \\ &= \frac{\pi_k N(x_n | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j N(x_n | \mu_j, \Sigma_j)} \end{aligned} \quad (37)$$

Vamos interpretar π_k como a probabilidade de que o ponto x_n tenha sido gerado pelo cluster k **a posteriori** e $\gamma(z_{nk})$ como a probabilidade de que o ponto x_n pertença ao cluster k **a posteriori**. Também podemos chamar $\gamma(z_{nk})$ de *responsabilidade* do cluster k pelo ponto x_n , pois ela indica o quanto o cluster k é responsável por gerar o ponto x_n .

3.2 Máxima Verossimilhança

Suponha que temos um conjunto de dados $X = \{x_1, x_2, \dots, x_N\}$ com $x_i \in \mathbb{R}^D$ e queremos modelar essa matriz $N \times D$ como uma mistura de K gaussianas. As variáveis latentes $Z = \{z_1, z_2, \dots, z_N\}$ que indicam de qual cluster cada ponto foi gerado também serão representadas por uma matriz $N \times K$ de vetores one-hot. A função de log-verossimilhança do modelo é então dada por:

$$\ln p(X \mid \mu, \Sigma, \pi) = \sum_{n=1}^N \ln p(x_n \mid \mu, \Sigma, \pi) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k N(x_n \mid \mu_k, \Sigma_k) \right\} \quad (38)$$

Acaba que maximizar essa verossimilhança diretamente é difícil, pois a presença da soma dentro do log torna a derivada complicada. Uma alternativa válida é maximizar a verossimilhança por métodos de otimização de gradiente, porém, nós vamos utilizar o algoritmo Expectation-Maximization (EM), que é um método iterativo para encontrar estimativas de máxima verossimilhança em modelos com variáveis latentes.

3.3 Expectation-Maximization (EM) para GMMs

Para facilitar o entendimento das contas, defina $N_k = \sum_{n=1}^N \gamma(z_{nk})$

Teorema 3.3.1: Fixando os parâmetros do modelo e variando apenas μ , o valor ótimo de μ_k é dado por:

$$\mu_k = \frac{\sum_{n=1}^N \gamma(z_{nk}) x_n}{\sum_{n=1}^N \gamma(z_{nk})} = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) x_n \quad (39)$$

Demonstração: Para encontrar o valor ótimo de μ_k , derivamos a função de log-verossimilhança em relação a μ_k e igualamos a zero:

$$\frac{\partial \ln p(X \mid \mu, \Sigma, \pi)}{\partial \mu_k} = \sum_{n=1}^N \frac{\pi_k N(x_n \mid \mu_k, \Sigma_k)}{\underbrace{\sum_{j=1}^K \pi_j N(x_n \mid \mu_j, \Sigma_j)}_{\gamma(z_{nk})}} \frac{1}{N(x_n \mid \mu_k, \Sigma_k)} \frac{\partial N(x_n \mid \mu_k, \Sigma_k)}{\partial \mu_k} = 0 \quad (40)$$

A derivada da densidade gaussiana em relação a μ_k é dada por:

$$\frac{\partial N(x_n \mid \mu_k, \Sigma_k)}{\partial \mu_k} = N(x_n \mid \mu_k, \Sigma_k) \Sigma_k^{-1} (x_n - \mu_k) \quad (41)$$

Substituindo isso na equação anterior, obtemos:

$$\sum_{n=1}^N \gamma(z_{nk}) \Sigma_k^{-1} (x_n - \mu_k) = 0 \quad (42)$$

Multiplicando ambos os lados por Σ_k , obtemos:

$$\sum_{n=1}^N \gamma(z_{nk}) (x_n - \mu_k) = 0 \quad (43)$$

Rearranjando os termos, obtemos:

$$\mu_k \sum_{n=1}^N \gamma(z_{nk}) = \sum_{n=1}^N \gamma(z_{nk}) x_n \quad (44)$$

Dividindo ambos os lados por $\sum_{n=1}^N \gamma(z_{nk})$, obtemos a expressão desejada para μ_k . $\square$

Teorema 3.3.2: Fixando os parâmetros do modelo e variando apenas Σ , o valor ótimo de Σ_k é dado por:

$$\Sigma_k = \frac{\sum_{n=1}^N \gamma(z_{nk}) (x_n - \mu_k)(x_n - \mu_k)^T}{\sum_{n=1}^N \gamma(z_{nk})} = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) (x_n - \mu_k)(x_n - \mu_k)^T \quad (45)$$

Demonstração: Para encontrar o valor ótimo de Σ_k , derivamos a função de log-verossimilhança em relação a Σ_k e igualamos a zero:

$$\frac{\partial \ln p(X | \mu, \Sigma, \pi)}{\partial \Sigma_k} = \sum_{n=1}^N \underbrace{\frac{\pi_k N(x_n | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j N(x_n | \mu_j, \Sigma_j)}}_{\gamma(z_{nk})} \frac{1}{N(x_n | \mu_k, \Sigma_k)} \frac{\partial N(x_n | \mu_k, \Sigma_k)}{\partial \Sigma_k} = 0 \quad (46)$$

A derivada da densidade gaussiana em relação a Σ_k é dada por:

$$\frac{\partial N(x_n | \mu_k, \Sigma_k)}{\partial \Sigma_k} = \frac{1}{2} N(x_n | \mu_k, \Sigma_k) (\Sigma_k^{-1} (x_n - \mu_k)(x_n - \mu_k)^T \Sigma_k^{-1} - \Sigma_k^{-1}) \quad (47)$$

Substituindo isso na equação anterior, obtemos:

$$\sum_{n=1}^N \gamma(z_{nk}) (\Sigma_k^{-1} (x_n - \mu_k)(x_n - \mu_k)^T \Sigma_k^{-1} - \Sigma_k^{-1}) = 0 \quad (48)$$

Multiplicando ambos os lados por Σ_k , obtemos:

$$\sum_{n=1}^N \gamma(z_{nk}) ((x_n - \mu_k)(x_n - \mu_k)^T \Sigma_k^{-1} - I) = 0 \quad (49)$$

Rearranjando os termos, obtemos:

$$\sum_{n=1}^N \gamma(z_{nk}) (x_n - \mu_k)(x_n - \mu_k)^T = \sum_{n=1}^N \gamma(z_{nk}) \Sigma_k \quad (50)$$

Dividindo ambos os lados por $\sum_{n=1}^N \gamma(z_{nk})$, obtemos a expressão desejada para Σ_k . $\square$

Teorema 3.3.3: Fixando os parâmetros do modelo e variando apenas π , o valor ótimo de π_k é dado por:

$$\pi_k = \frac{\sum_{n=1}^N \gamma(z_{nk})}{N} = \frac{N_k}{N} \quad (51)$$

Demonstração: Sabendo que $\sum_k \pi_k = 1$, usamos de multiplicadores de lagrange para encontrar o valor ótimo de π_k . Definimos a função lagrangiana como:

$$L(\pi, \lambda) = \ln p(X \mid \mu, \Sigma, \pi) + \lambda \left(\sum_{k=1}^K \pi_k - 1 \right) \quad (52)$$

Derivando em relação a π_k e igualando a zero, obtemos:

$$\frac{\partial L}{\partial \pi_k} = \frac{\gamma(z_{nk})}{\pi_k} + \lambda = 0 \quad (53)$$

Isolando π_k , obtemos:

$$\pi_k = -\frac{\lambda}{\gamma(z_{nk})} \quad (54)$$

Usando a condição de normalização $\sum_k \pi_k = 1$, podemos encontrar o valor de λ :

$$\sum_{k=1}^K -\frac{\lambda}{\gamma(z_{nk})} = 1 \quad (55)$$

Resolvendo para λ , obtemos:

$$\lambda = -\frac{1}{\sum_{k=1}^K \frac{1}{\gamma(z_{nk})}} \quad (56)$$

Substituindo esse valor de λ na expressão para π_k , obtemos:

$$\pi_k = \frac{\gamma(z_{nk})}{\sum_{j=1}^K \gamma(z_{nj})} = \frac{N_k}{N} \quad (57)$$

□

Vale ressaltar que o Teorema 3.3.1, Teorema 3.3.2 e Teorema 3.3.3 não representam formas fechadas dos parâmetros do modelo, pois eles dependem de $\gamma(z_{nk})$, que por sua vez depende dos próprios parâmetros do modelo. Portanto, não podemos resolver essas equações diretamente. Em vez disso, usamos o algoritmo EM, que alterna entre calcular $\gamma(z_{nk})$ com os parâmetros atuais (passo E) e atualizar os parâmetros do modelo usando as fórmulas acima (passo M).

Algoritmo EM

```

1 function EM(X) {
2   initialize  $\mu_k, \Sigma_k, \pi_k$ 
3   // Passo E
4    $\gamma(z_{nk}) = \frac{\pi_k N(x_n \mid \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j N(x_n \mid \mu_j, \Sigma_j)}$ 
5   // Passo M
6    $N_k = \sum_{n=1}^N \gamma(z_{nk})$ 
7    $\mu_k = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) x_n$ 
8    $\Sigma_k = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) (x_n - \mu_k)(x_n - \mu_k)^T$ 
9    $\pi_k = \frac{N_k}{N}$ 
10  // Calcular a log-verossimilhança
11   $\ln p(X \mid \mu, \Sigma, \pi) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k N(x_n \mid \mu_k, \Sigma_k) \right\}$ 
12 }
```

3.4 Algoritmo EM Variacional

O Algoritmo EM que vimos agora é uma aplicação de uma versão mais geral do algoritmo EM. Ele tem como objetivo achar a verossimilhança de modelos com variáveis latentes

Representamos o conjunto de dados por uma matriz X onde a n -ésima linha é representada por x_n^T . De forma similar, definimos o conjunto de variáveis latentes como uma matriz Z onde a n -ésima linha é representada por z_n^T . A função de log-verossimilhança do modelo é então dada por:

$$\ln p(X | \theta) = \ln \left\{ \sum_Z p(X, Z | \theta) \right\} \quad (58)$$

Note que nossa discussão também se aplica com variáveis latentes contínuas trocando a soma interna por uma integral. O problema central aqui é que o somatório/integral no interior do log torna a maximização da verossimilhança difícil. Chamamos o conjunto $\{X, Z\}$ de **dataset completo**, enquanto chamamos o conjunto $\{X\}$ de **dataset incompleto**. O problema é que não podemos observar Z , nosso conhecimento sobre as variáveis latentes se dá apenas a partir da posteriori $p(Z | X, \theta)$. Como não conseguimos olhar diretamente para $\ln p(X, Z | \theta)$, então consideramos seu valor esperado sobre a distribuição posteriori de Z dado X e os parâmetros atuais $\theta^{(t)}$. O foco desse capítulo não é dar uma derivação formal do algoritmo EM, porém, vamos deixar o framework geral escrito e mostrar ele sendo aplicado novamente ao caso das GMMs.

Algoritmo EM (Geral)

```

1 function EM( $X$ ) {
2   initialize  $\theta^{(0)}$ 
3   repeat {
4     // Passo E
5     calcular  $p(Z | X, \theta^{(t)})$ 
6     // Passo M
7      $Q(\theta, \theta^{(t)}) = \sum_z p(z | X, \theta^{(t)}) \ln p(X, z | \theta)$ 
8      $\theta^{(t+1)} = \operatorname{argmax}_{\theta} Q(\theta, \theta^{(t)})$ 
9     // Checando se convergiu
10    if not converged yet {
11      |  $\theta^{(t)} = \theta^{(t+1)}$ 
12    }
13  }
```

Revisitando o caso das GMMs, vamos primeiro considerar o problema de maximizar a verossimilhança do dataset completo, que é dado por:

$$p(X, Z | \mu, \Sigma, \pi) = \prod_{n=1}^N \prod_{k=1}^K \{\pi_k N(x_n | \mu_k, \Sigma_k)\}^{z_{nk}} \quad (59)$$

aplicando log

$$\ln p(X, Z | \mu, \Sigma, \pi) = \sum_{n=1}^N \sum_{k=1}^K z_{nk} \{\ln \pi_k + \ln N(x_n | \mu_k, \Sigma_k)\} \quad (60)$$

podemos ver que, em comparação com a equação (58), o log da verossimilhança tem o somatório do lado de fora, o que facilita a derivada. O problema é que não podemos observar Z , então vamos considerar o valor esperado do log da verossimilhança do dataset completo sobre a distribuição posteriori de Z dado X .

Pelo teorema de bayes, vamos chegar que a posteriori é obtida com:

$$p(Z | X, \mu, \Sigma, \pi) \propto p(X, Z | \mu, \Sigma, \pi) = \prod_{n=1}^N \prod_{k=1}^K \{\pi_k N(x_n | \mu_k, \Sigma_k)\}^{z_{nk}} \quad (61)$$

perceba que isso mostra que cada z_n é independente dos outros z_m dado x_n . Então podemos escrever a média de z_{nk} sobre o regime da posteriori como:

$$\begin{aligned} \mathbb{E}_{z_{nk} \sim p(z_n | x_n, \dots)}[z_{nk}] &= \frac{\sum_{z_{nk}} z_{nk} [\pi_k N(x_n | \mu_k, \Sigma_k)]^{z_{nk}}}{\sum_{z_{nj}} [\pi_j N(x_n | \mu_j, \Sigma_j)]^{z_{nj}}} \\ &= \frac{\pi_k N(x_n | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j N(x_n | \mu_j, \Sigma_j)} = \gamma(z_{nk}) \end{aligned} \quad (62)$$

Então vamos ter que a esperança da log-verossimilhança do dataset completo sobre a distribuição posteriori de Z dado X é:

$$\mathbb{E}_{Z \sim p(Z | X, \mu, \Sigma, \pi)}[\ln p(X, Z | \mu, \Sigma, \pi)] = \sum_{n=1}^N \sum_{k=1}^K \gamma(z_{nk}) \{\ln \pi_k + \ln N(x_n | \mu_k, \Sigma_k)\} \quad (63)$$

Dado essa equação, podemos fixar valores iniciais para μ , Σ e π para calcular $\gamma(z_{nk})$, depois maximizamos os valores dos parâmetros do modelo com base na equação (63) com as fórmulas já vistas anteriormente no Teorema 3.3.1, Teorema 3.3.2 e Teorema 3.3.3 e repetimos esse processo até que a convergência seja alcançada. Esse é o algoritmo EM aplicado aos GMMs.

3.5 Bernoulli Mixture Models

Agora que vimos os GMMs e como derivar o algoritmo de resolução do problema, que tal analisarmos o caso de variáveis com distribuições discretas? Vamos considerar o caso de variáveis binárias, que podem ser modeladas com distribuições de Bernoulli. Esse modelo também é conhecido como **Análise de Classe Latentes**

Definição 3.5.1 (Vetor Bernoulli): Considere um conjunto de D variáveis aleatórias binárias $X = \{x_1, x_2, \dots, x_D\}$, onde cada variável x_i segue uma distribuição de Bernoulli com parâmetro μ_i , ou seja, $\mathbb{P}(x_i = 1) = \mu_i$ e $\mathbb{P}(x_i = 0) = 1 - \mu_i$. O vetor aleatório x é chamado de vetor Bernoulli e sua função de probabilidade conjunta é dada por:

$$p(x | \mu) = \prod_{i=1}^D \mu_i^{x_i} (1 - \mu_i)^{1-x_i} \quad (64)$$

onde $x \in \mathbb{R}^D$ e $\mu \in \mathbb{R}^D$

Teorema 3.5.1 (Validade da distribuição): Seja $p(x|\mu)$ um Modelo de Mistura de Bernoulli. Então, $p(x|\mu)$ é uma distribuição de probabilidade válida, ou seja, $p(x|\mu) \geq 0$ para todo x e $\sum_{x \in \{0,1\}^D} p(x|\mu) = 1$.

Demonstração: Para mostrar que $p(x|\mu) \geq 0$, note que cada termo na multiplicação é não-negativo, pois $\mu_i^{x_i} \geq 0$ e $(1 - \mu_i)^{1-x_i} \geq 0$. Portanto, $p(x|\mu) \geq 0$ para todo x .

Para mostrar que a soma de $p(x|\mu)$ sobre todos os possíveis vetores binários de dimensão D é igual a 1, usamos a propriedade da distribuição de Bernoulli:

$$\sum_{x \in \{0,1\}^D} p(x|\mu) = \sum_{x \in \{0,1\}^D} \prod_{i=1}^D \mu_i^{x_i} (1 - \mu_i)^{1-x_i} \quad (65)$$

Como cada termo na multiplicação é independente dos outros termos, podemos reescrever a soma como um produto de somas:

$$= \prod_{i=1}^D \sum_{x_i \in \{0,1\}} \mu_i^{x_i} (1 - \mu_i)^{1-x_i} \quad (66)$$

Cada soma interna é igual a 1, pois:

$$\sum_{x_i \in \{0,1\}} \mu_i^{x_i} (1 - \mu_i)^{1-x_i} = \mu_i + (1 - \mu_i) = 1 \quad (67)$$

Portanto, temos:

$$\sum_{x \in \{0,1\}^D} p(x|\mu) = \prod_{i=1}^D 1 = 1 \quad (68)$$

□

Conseguimos ver também que:

$$\begin{aligned} \mathbb{E}[x] &= \mu \\ \text{Cov}[x] &= \text{diag}(\mu_i(1 - \mu_i)) \end{aligned} \quad (69)$$

Definição 3.5.2 (Modelo de Mistura de Bernoulli): Um Modelo de Mistura de Bernoulli é definido como:

$$p(x|\mu, \pi) = \sum_{k=1}^K \pi_k p(x|\mu_k) = \sum_{k=1}^K \pi_k \prod_{i=1}^D \mu_{ki}^{x_i} (1 - \mu_{ki})^{1-x_i} \quad (70)$$

onde π_k são os pesos dos clusters, que satisfazem $\sum_{k=1}^K \pi_k = 1$, e μ_{ki} são os parâmetros da distribuição de Bernoulli para o cluster k .

Teorema 3.5.2 (Validade da distribuição): Seja $p(x|\mu, \pi)$ um Modelo de Mistura de Bernoulli com K componentes. Então, $p(x|\mu, \pi)$ é uma distribuição de probabilidade válida, ou seja, $p(x|\mu, \pi) \geq 0$ para todo x e $\sum_{x \in \{0,1\}^D} p(x|\mu, \pi) = 1$.

Demonstração: Para mostrar que $p(x|\mu, \pi) \geq 0$, note que cada termo na soma é não-negativo, pois $\pi_k \geq 0$ e $p(x|\mu_k) \geq 0$. Portanto, $p(x|\mu, \pi) \geq 0$ para todo x .

Para mostrar que a soma de $p(x|\mu, \pi)$ sobre todos os possíveis vetores binários de dimensão D é igual a 1, usamos a linearidade da soma:

$$\sum_{x \in \{0,1\}^D} p(x|\mu, \pi) = \sum_{x \in \{0,1\}^D} \sum_{k=1}^K \pi_k p(x|\mu_k) \quad (71)$$

Podemos trocar a ordem das somas:

$$= \sum_{k=1}^K \pi_k \sum_{x \in \{0,1\}^D} p(x|\mu_k) \quad (72)$$

Cada soma interna é igual a 1, pois $p(x|\mu_k)$ é uma distribuição de probabilidade válida. Portanto, temos:

$$= \sum_{k=1}^K \pi_k * 1 = \sum_{k=1}^K \pi_k = 1 \quad (73)$$

□

A média e covariância dessa distribuição de mistura é dada por:

$$\begin{aligned} \mathbb{E}[x] &= \sum_{k=1}^K \pi_k \mu_k \\ \text{Cov}[x] &= \sum_{k=1}^K \pi_k (\Sigma_k + \mu_k \mu_k^T) - \mathbb{E}[x] \mathbb{E}[x]^T \end{aligned} \quad (74)$$

onde $\Sigma_k = \text{diag}(\mu_{ki}(1 - \mu_{ki}))$. Dado um conjunto de dados $X = \{x_1, \dots, x_N\}$ que segue o modelo de mistura de Bernoulli, a função de log-verossimilhança é dada por:

$$\ln p(X | \mu, \pi) = \sum_{n=1}^N \ln p(x_n | \mu, \pi) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k p(x_n | \mu_k) \right\} \quad (75)$$

vemos novamente a mesma dificuldade de maximizar a verossimilhança diretamente, então vamos encontrar as fórmulas de atualização para o algoritmo EM aplicado ao modelo de mistura de Bernoulli.

Para isso, definimos, assim como no caso de mistura de gaussianas, a variável latente z_n que indica de qual cluster o ponto x_n foi gerado. A distribuição condicional de x_n dado z_n é dada por:

$$p(x_n | z_n, \mu) = \prod_{k=1}^K p(x|\mu_k)^{z_{nk}} \quad (76)$$

e priori aqui é a mesma usada no caso de mistura de gaussianas:

$$p(z_n | \pi) = \prod_{k=1}^K \pi_k^{z_{nk}} \quad (77)$$

Antes de enunciarmos os teoremas com os valores ótimos, precisamos escrever a função de log-verossimilhança do dataset completo, que é dada por:

$$\ln p(X, Z | \mu, \pi) = \sum_{n=1}^N \sum_{k=1}^K z_{nk} \left\{ \ln \pi_k + \sum_{i=1}^D [x_{ni} \ln \mu_{ki} + (1 - x_{ni}) \ln(1 - \mu_{ki})] \right\} \quad (78)$$

E a esperança da log-verossimilhança do dataset completo sobre a distribuição posteriori de Z dado X é:

$$\begin{aligned} \mathbb{E}_{Z \sim p(Z|X, \mu, \pi)} [\ln p(X, Z | \mu, \pi)] &= \sum_{n=1}^N \sum_{k=1}^K \gamma(z_{nk}) \left\{ \ln \pi_k \right. \\ &\quad \left. + \sum_{i=1}^D [x_{ni} \ln \mu_{ki} + (1 - x_{ni}) \ln(1 - \mu_{ki})] \right\} \end{aligned} \quad (79)$$

onde $\gamma(z_{nk})$ é a probabilidade de $z_{nk} = 1$ sob a distribuição posteriori. No passo **E** do algoritmo, isso é calculado com o teorema de bayes:

$$\gamma(z_{nk}) = \frac{p(z_{nk} = 1)p(x_n | z_{nk} = 1, \mu_k)}{p(x_n | \mu, \pi)} = \frac{\pi_k p(x_n | \mu_k)}{\sum_{j=1}^K \pi_j p(x_n | \mu_j)} \quad (80)$$

Teorema 3.5.3 (Valor ótimo de μ): Fixando os parâmetros do modelo e variando apenas μ , o valor ótimo de μ_{ki} é dado por:

$$\mu_{ki} = \frac{\sum_{n=1}^N \gamma(z_{nk}) x_{ni}}{\sum_{n=1}^N \gamma(z_{nk})} = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) x_{ni} \quad (81)$$

ou

$$\mu_k = \frac{\sum_{n=1}^N \gamma(z_{nk}) x_n}{\sum_{n=1}^N \gamma(z_{nk})} = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) x_n \quad (82)$$

Demonstração: Para encontrar o valor ótimo de μ_{ki} , derivamos a função de log-verossimilhança em relação a μ_{ki} e igualamos a zero:

$$\frac{\partial \ln p(X, Z | \mu, \pi)}{\partial \mu_{ki}} = \sum_{n=1}^N \gamma(z_{nk}) \frac{x_{ni} - \mu_{ki}}{\mu_{ki}(1 - \mu_{ki})} = 0 \quad (83)$$

Rearranjando os termos, obtemos:

$$\sum_{n=1}^N \gamma(z_{nk}) x_{ni} = \mu_{ki} \sum_{n=1}^N \gamma(z_{nk}) \quad (84)$$

Dividindo ambos os lados por $\sum_{n=1}^N \gamma(z_{nk})$, obtemos a expressão desejada para μ_{ki} . $\square$

Teorema 3.5.4 (Valor ótimo de π): Fixando os parâmetros do modelo e variando apenas π , o valor ótimo de π_k é dado por:

$$\pi_k = \frac{\sum_{n=1}^N \gamma(z_{nk})}{N} = \frac{N_k}{N} \quad (85)$$

Demonstração: Sabendo que $\sum_k \pi_k = 1$, usamos de multiplicadores de lagrange para encontrar o valor ótimo de π_k . Definimos a função lagrangiana como:

$$L(\pi, \lambda) = \ln p(X, Z \mid \mu, \pi) + \lambda \left(\sum_{k=1}^K \pi_k - 1 \right) \quad (86)$$

Derivando em relação a π_k e igualando a zero, obtemos:

$$\frac{\partial L}{\partial \pi_k} = \frac{\gamma(z_{nk})}{\pi_k} + \lambda = 0 \quad (87)$$

Isolando π_k , obtemos:

$$\pi_k = -\frac{\lambda}{\gamma(z_{nk})} \quad (88)$$

Usando a condição de normalização $\sum_k \pi_k = 1$, podemos encontrar o valor de λ :

$$\sum_{k=1}^K -\frac{\lambda}{\gamma(z_{nk})} = 1 \quad (89)$$

Resolvendo para λ , obtemos:

$$\lambda = -\frac{1}{\sum_{k=1}^K \frac{1}{\gamma(z_{nk})}} \quad (90)$$

Substituindo esse valor de λ na expressão para π_k , obtemos:

$$\pi_k = \frac{\gamma(z_{nk})}{\sum_{j=1}^K \gamma(z_{nj})} = \frac{N_k}{N} \quad (91)$$

□

3.6 Singularidades e Identificabilidade

É válido ressaltar a existência desse problema, que é intrínseco do algoritmo que utilizamos (variáveis latentes) no problema de mistura de gaussianas. Para simplicidade e ilustrar o problema (também se aplica à casos mais gerais), considere uma mistura de gaussianas cujos componentes de covariância são matrizes escalares, ou seja, $\Sigma_k = \sigma_k^2 I$. Suponha também que um dos componentes da mistura (digamos, o j -ésimo) tem sua média μ_j exatamente igual a algum dos pontos do banco ($x_n = \mu_j$). Esse ponto então vai contribuir para a verossimilhança um termo:

$$N(x_n \mid \mu_j, \Sigma_j) = \frac{1}{(2\pi)^{\frac{1}{2}} \sigma_j} \quad (92)$$

se considerarmos $\sigma_j \rightarrow 0$, então o termo vai para ∞ assim como a verossimilhança. Ou seja, o problema de maximização da log-verossimilhança não é bem definido, pois não existe um máximo global. Esse problema é conhecido como **singularidade** e é um problema clássico do algoritmo EM aplicado a modelos de mistura de gaussianas.

Outro problema é que, dado um ponto (não-degenerado) no espaço dos parâmetros, existem permutações dos parâmetros que geram a mesma distribuição de probabilidade. Por exemplo, considere uma mistura de duas gaussianas com parâmetros μ_1, Σ_1, π_1 e μ_2, Σ_2, π_2 . Se permutarmos os índices das gaussianas, ou seja, trocarmos μ_1 com μ_2 , Σ_1 com Σ_2 e π_1 com π_2 , a distribuição de probabilidade gerada será a mesma. Esse problema é conhecido como **identificabilidade** e é um problema clássico do algoritmo EM aplicado a modelos de mistura de gaussianas.

Variational Autoencoders

4.1 Introdução

Autoencoders são modelos de aprendizado não supervisionado projetados para aprender representações compactas dos dados. Seu objetivo é comprimir uma entrada em uma representação de menor dimensão e, em seguida, reconstruir a entrada original a partir dessa representação.

A arquitetura de um autoencoder é composta por duas partes principais:

- **Encoder:** transforma a entrada original em uma representação latente, também chamada de **código** ou **embedding**.
- **Decoder:** utiliza essa representação latente para reconstruir uma aproximação da entrada original.

De forma simplificada, dado um dado de entrada (x), o encoder produz uma representação (z),

$$z = f(x), \quad (93)$$

e o decoder gera uma reconstrução ($\hat{x}$),

$$\hat{x} = g(z). \quad (94)$$

Durante o treinamento, os parâmetros do modelo são ajustados para minimizar a diferença entre (x) e ($\hat{x}$), fazendo com que a representação latente retenha as características mais relevantes dos dados.

Ao aprender a reconstruir as entradas a partir de uma representação comprimida, os autoencoders podem descobrir estruturas e padrões presentes nos dados, sendo amplamente utilizados para redução de dimensionalidade, compressão, remoção de ruído, detecção de anomalias e aprendizado de representações.

4.2 Autoencoders Determinísticos

Esses autoencoders são versões clássicas e mais simples. Eles consistem em uma rede neural que aprende a mapear entradas para saídas, passando por uma camada intermediária de menor dimensão. O objetivo é minimizar a diferença entre a entrada e a saída reconstruída, geralmente utilizando funções de perda como o erro quadrático médio (MSE).

4.2.1 Autoencoders Profundos

A principal ideia desse autoencoder é uma rede neural que recebe como input um vetor $x \in \mathbb{R}^D$, passa ele por diversas camadas ocultas de menor dimensão e tenta, a partir de um novo vetor $z \in \mathbb{R}^M$ ($M < D$) reconstruir o vetor original x . A função de perda utilizada nesses autoencoders é dada por:

$$E(w) = \frac{1}{2} \sum_{n=1}^N \|x_n - y(x_n, w)\|^2 \quad (95)$$

onde $y(x_n, w)$ é a saída da rede neural com pesos w para a entrada x_n . A função de perda é minimizada utilizando o algoritmo de retropropagação (backpropagation) e métodos de otimização como o gradiente descendente.

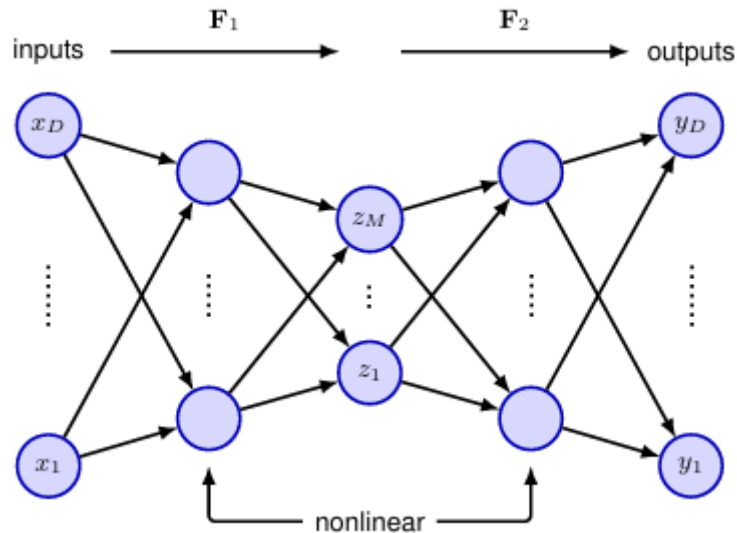


Figura 5: Arquitetura de um autoencoder profundo. A entrada x é comprimida em uma representação latente z e, em seguida, reconstruída como $y(x, w)$.

Como podemos ver na Figura 5, a arquitetura do autoencoder profundo pode ser interpretada como dois mapeamentos distintos F_1 e F_2 , onde F_1 é o encoder que mapeia a entrada x para a representação latente z , e F_2 é o decoder que mapeia a representação latente z de volta para a reconstrução da entrada original $y(x, w)$. A função de perda é então minimizada ajustando os pesos da rede neural para melhorar a qualidade da reconstrução.

4.2.2 Autoencoders Esparsos

Uma forma tradicional de limitar a capacidade de um autoencoder consiste em utilizar uma representação latente de dimensão menor que a dimensão dos dados de entrada. Entretanto, essa não é a única maneira de impor uma representação compacta. Nos **autoencoders esparsos**, em vez de restringir o número de neurônios da camada latente, utiliza-se uma regularização que incentiva apenas uma pequena fração desses neurônios a permanecer ativa para cada exemplo.

A ideia é permitir que a camada latente possua muitas unidades, mas forçar a maioria delas a assumir valores nulos ou próximos de zero. Dessa forma, cada amostra é representada por apenas alguns neurônios ativos, produzindo uma representação de baixa dimensionalidade efetiva.

Uma forma simples de obter esse comportamento é adicionar uma penalização L_1 sobre as ativações da camada latente. A função de custo passa a ser dada por

$$E(w) = \tilde{E}(w) + \lambda \sum_{m=1}^M |z_m|, \quad (96)$$

onde $\tilde{E}(w)$ representa o erro de reconstrução, z_m corresponde à ativação do neurônio latente m , e λ controla a intensidade da regularização.

Como a norma L_1 favorece soluções esparsas, o treinamento passa a buscar simultaneamente uma boa reconstrução dos dados e uma representação latente na qual poucos neurônios estejam ativos. Em consequência, o modelo é capaz de aprender características relevantes dos dados mesmo quando a camada latente possui um número elevado de unidades.

4.2.3 Denoising Autoencoders

Vimos que para o autoencoder aprender representações úteis, é necessário impor restrições à sua capacidade de reconstrução. Uma abordagem alternativa é treinar o autoencoder para

reconstruir a entrada original a partir de uma versão corrompida dela. Essa técnica é conhecida como **denoising autoencoder**. Assim, intuitivamente, eu forço o meu autoencoder a aprender representações robustas dos dados, que capturam as características essenciais e ignoram o ruído.

$$E(w) = \frac{1}{2} \sum_{n=1}^N \|x_n - y(\tilde{x}_n, w)\|^2 \quad (97)$$

Um método de impor ruído nas entradas é selecionar uma fração $\tau \in (0, 1)$ das amostras e colocar parte de suas entradas como 0. Por exemplo, se $\tau = 0.2$, então 20% das entradas de cada amostra selecionada serão corrompidas, ou seja, substituídas por zero. Outro método é adicionar ruído gaussiano às entradas, ou seja, para cada entrada x_n , adicionamos um ruído ε proveniente de uma distribuição normal com média zero e desvio padrão σ , resultando em uma entrada corrompida $\tilde{x}_n = x_n + \varepsilon$.

4.3 Autoencoders Variacionais

Agora chegamos na brincadeira de gente grande. Nós já vimos que, a função de verossimilhança de um modelo com variáveis latentes dada por:

$$p(x|w) = \int p(x|z, w)p(z) dz \quad (98)$$

onde $p(x|z, w)$ é definida por uma rede neural profunda, é intratável pois a integral em z não tem forma fechada. Os autoencoders variacionais (VAEs) resolvem esse problema utilizando uma aproximação variacional para a posteriori $p(z|x, w)$, que é definida por outra rede neural profunda. A ideia é otimizar os parâmetros do modelo para maximizar a verossimilhança dos dados, enquanto simultaneamente aproximamos a posteriori das variáveis latentes. Existem 3 conceitos-chave dentro dos VAEs:

1. Utilizar o **ELBO** (Evidence Lower Bound) para aproximar a verossimilhança dos dados
2. **Inferência Amortizada** onde um segundo modelo, a **rede encoder**, é usada para aproximar a distribuição posteriori das variáveis latentes no passo **E** em vez de calcular para cada ponto
3. Fazer o treino da rede encoder tratável utilizando do **truque da reparametrização**

Considere um modelo generativo com distribuição $p(x|z, w)$ governado pela saída de uma rede $g(w, z)$ (Por exemplo, $g(w, z)$ retorna a média de uma distribuição gaussiana). Considere também $p(z) \sim N(0, I)$ sobre $z \in \mathbb{R}^M$

Lembrando do documento da A1, vimos que:

$$\ln p(x|w) = \mathcal{L}(w) + \text{KL}(q(z) \parallel p(z|x, w)) \quad (99)$$

onde $\mathcal{L}(w)$ é o ELBO e $q(z)$ é a distribuição aproximada das variáveis latentes e $\mathcal{L}(w)$ é dado por:

$$\mathcal{L}(w) = \int q(z) \ln \frac{p(x|z, w)p(z)}{q(z)} dz \quad (100)$$

e $\text{KL}(p \parallel q)$ é definido como:

$$\text{KL}(p \parallel q) = \int p(z) \ln \frac{p(z)}{q(z)} dz \quad (101)$$

Pelo que tínhamos visto no primeiro documento, sabemos que $\ln p(x|w) \geq \mathcal{L}$ (Por isso o nome **EVIDENCE LOWER BOUND**). Mesmo que $\ln p(x|w)$ seja intratável, podemos aproximar ela utilizando de $\mathcal{L}$ aproximado por **monte carlo**.

Considere o conjunto de dados $x_1, \dots, x_N$. Temos então que

$$\ln p(D|w) = \sum_{n=1}^N \mathcal{L}_n + \sum_{n=1}^N \text{KL}(q_n(z_n|x_n) \parallel p(z_n|x_n, w)) \quad (102)$$

onde

$$\mathcal{L}_n = \int q_n(z_n|x_n) \ln \left\{ \frac{p(x_n|z_n, w) \cdot p(z_n)}{q_n(z_n|x_n)} \right\} dz_n \quad (103)$$

Perceba que agora, cada x_n tem sua variável latente, o que indica que cada variável z_n tem sua distribuição $q_n(z_n|x_n)$. Como a equação (102) é mantida independente da escolha de $q_n(z_n|x_n)$, podemos escolher $q_n(z_n|x_n)$ para cada ponto x_n de forma a maximizar $\mathcal{L}_n$ ou, equivalentemente, minimizar $\text{KL}(q_n(z_n|x_n) \parallel p(z_n|x_n, w))$. EM GMMs, conseguimos achar $q_n(z_n|x_n)$ de forma exata ($q_n(z_n|x_n) = p(z_n|x_n, w)$)

$$p(z_n|x_n, w) = \frac{p(x_n|z_n, w)p(z_n)}{p(x_n|w)} \quad (104)$$

o numerador é trivial de calcular, mas o denominador é intratável. Então precisamos de um método diferente para aproximar $q_n(z_n|x_n)$.

4.3.1 Inferência Amortizada

Nessa abordagem, nós treinamos **uma única** rede neural para aproximar todas as posteriores $p(z_n|x_n, w)$, chamada de **Encoder Network**. Essa abordagem se chama **inferência amortizada** que requer um encoder que gera uma única distribuição $q(z|x, \varphi)$ condicionada em x , onde φ são os parâmetros da rede neural. Nessa abordagem, a função-objetivo (dada pelo ELBO) depende tanto de φ quanto de w , assim ela faz a otimização conjunta dos parâmetros com abordagens baseadas em gradiente.

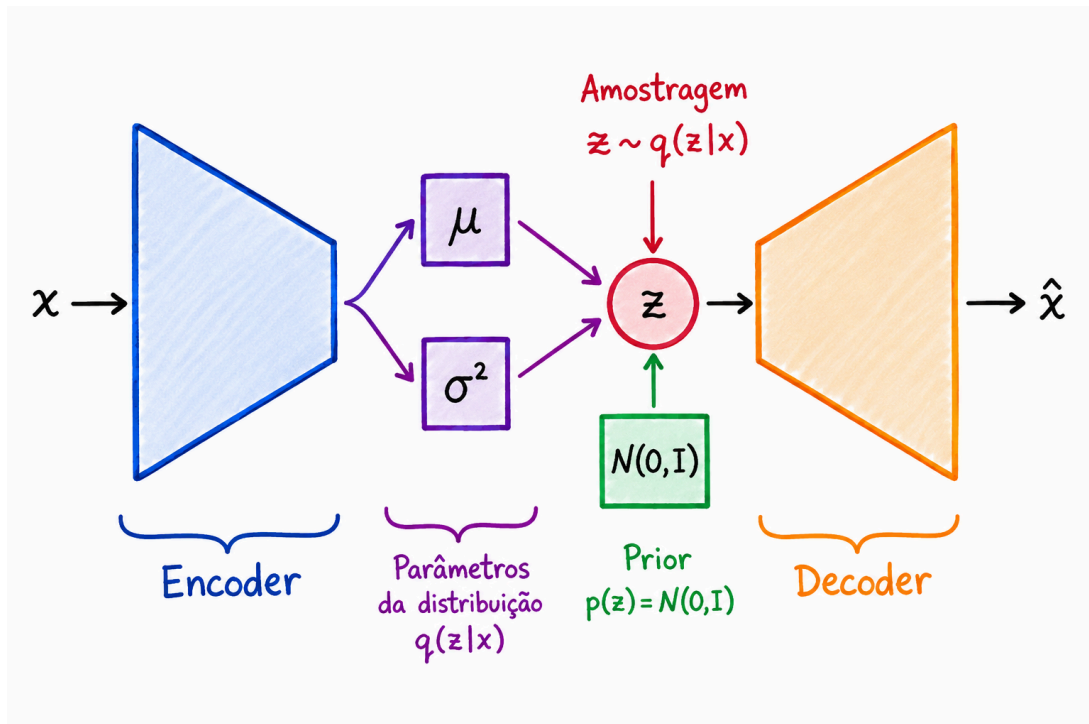


Figura 6: Arquitetura de um autoencoder variacional

Um encoder variacional é então composto por duas redes, um **encoder** que mapeia do espaço dos dados para um espaço latente e um **decoder** que mapeia do espaço latente de volta para o espaço dos dados e ambas as redes são treinadas simultaneamente para maximizar o ELBO.

Certo, mas agora temos que decidir ao menos qual espaço latente nós gostaríamos de mapear nossos dados pra termos uma base, correto? Sim! Uma escolha muito comum de se usar para o encoder é uma Gaussiana $N(\mu_j, \sigma_j^2 I)$ onde μ_j e σ_j são outputs de uma rede neural

$$q(z|x, \varphi) = \prod_{j=1}^M N(z_j | \mu_j(x, \varphi), \sigma_j^2(x, \varphi)) \quad (105)$$

4.3.2 Truque da Reparametrização

Infelizmente, temos que o lower bound ainda é intratável

$$\mathcal{L}_n(w, \varphi) = \int q(z_n|x_n, \varphi) \ln \left\{ \frac{p(x_n|z_n, w)p(z_n)}{q(z_n|x_n, \varphi)} \right\} dz_n \quad (106)$$

porque envolve integrar sobre as variáveis latentes $\{z\}$ e ele depende de forma complexa dos parâmetros da rede neural. Porém, podemos decompor essa integral em duas partes:

$$\mathcal{L}_n(w, \varphi) = \mathbb{E}_{z_n \sim q}[\ln p(x_n|z_n, w)] - \text{KL}(q(z_n|x_n, \varphi) \parallel p(z_n)) \quad (107)$$

Se nós escolhemos $q(z_n|x_n, \varphi)$ como uma Gaussiana, e $p(z_n)$ também, então a divergência KL entre elas tem uma forma fechada, que é dada por:

$$\text{KL}(q(z_n|x_n, \varphi) \parallel p(z_n)) = \frac{1}{2} \sum_{j=1}^M \{1 + \ln \sigma_j^2(x_n, \varphi) - \sigma_j^2(x_n, \varphi) - \mu_j^2(x_n, \varphi)\} \quad (108)$$

Já com relação à primeira parte, podemos tentar aproximar ela utilizando **monte carlo**

$$\mathbb{E}_{z_n \sim q}[\ln p(x_n|z_n, w)] = \int q(z_n|x_n, \varphi) \ln p(x_n|z_n, w) dz_n \approx \frac{1}{L} \sum_{l=1}^L \ln p(x_n|z_n^{(l)}, w) \quad (109)$$

onde $\{z_n^{(l)}\}$ são amostras de $q(z_n|x_n, \varphi)$. Toda essa equação (107) é diferenciável com relação a w , no entanto, ela tem uma relação complexa em φ para ser facilmente diferenciável.

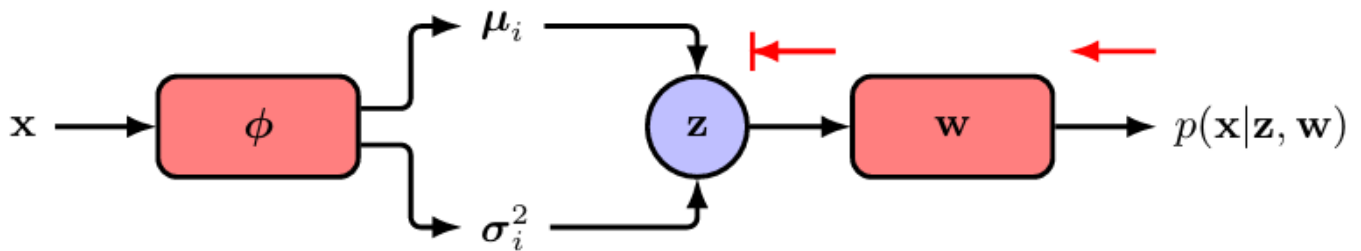


Figura 7: Esquema de como o erro se espalha no autoencoder. O fato de z depender de φ de forma complexa impede que o gradiente seja propagado através do processo de amostragem.

Para consertar isso, utilizamos do **truque da reparametrização**. Nessa abordagem, nós não vamos amostrar $z_n^{(l)}$ diretamente. Em vez disso, vamos amostrar $\varepsilon \sim N(0, 1)$. Após amostrar ε , nós podemos reparametrizar $z_n^{(l)}$ como:

$$z_{nj}^{(l)} = \mu(x_n, \varphi) + \sigma(x_n, \varphi) \cdot \varepsilon_{nj}^{(l)} \quad (110)$$

pois sabemos que $z_n^{(l)} \sim N(\mu(x_n, \varphi), \sigma^2(x_n, \varphi))$. Dessa forma, a amostragem de $z_n^{(l)}$ é feita de forma diferenciável com relação a φ , permitindo que o gradiente seja propagado através do processo de amostragem. Dessa forma, a função de erro do autoencoder variacional, depois de todas nossas premissas, é dada por:

$$\mathcal{L} = \sum_n \left\{ \frac{1}{2} \sum_{j=1}^M \{1 + \ln \sigma_{nj}^2 - \sigma_{nj}^2 - \mu_{nj}^2\} + \frac{1}{L} \sum_{l=1}^L \ln p(x_n | z_n^{(l)}, w) \right\} \quad (111)$$

onde, para simplificar a notação, nós escrevemos $\mu_{nj} = \mu_j(x_n, \varphi)$ e $\sigma_{nj} = \sigma_j(x_n, \varphi)$ e $z_n^{(l)} = \mu(x_n, \varphi) + \sigma(x_n, \varphi) \cdot \varepsilon^{(l)}$.

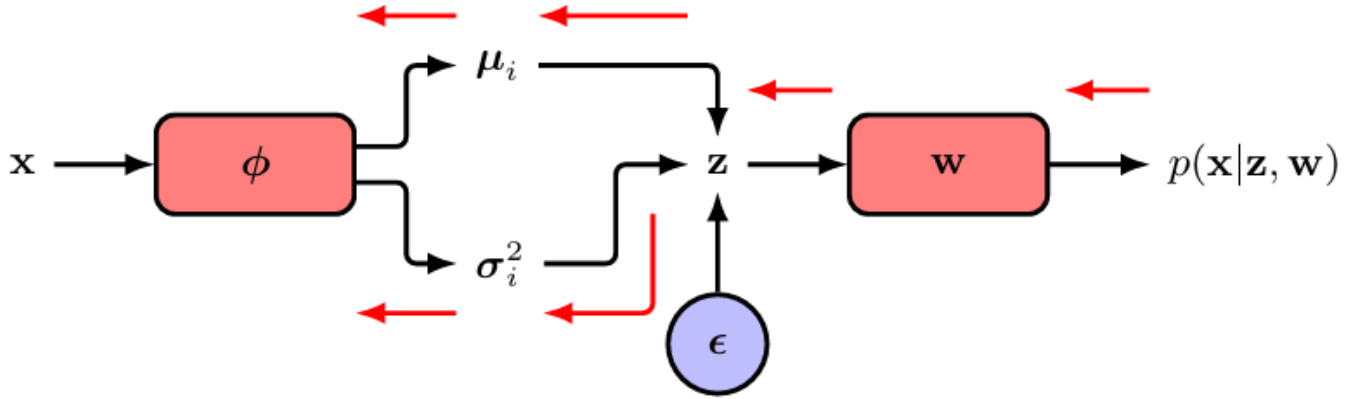


Figura 8: Esquema de como o erro se espalha no autoencoder após o truque da reparametrização. Como a amostragem não depende mais de φ , o gradiente pode ser propagado através do processo de amostragem.

Treinamento do VAE

```

1 function train_VAE( $D, w, \varphi$ ) {
2   for  $x_n \in D$  {
3      $\mathcal{L} \leftarrow 0$ 
4     for  $j \in \{1, 2, \dots, M\}$  {
5        $\varepsilon_{nj} \sim N(0, 1)$ 
6        $z_{nj} \leftarrow \mu_{nj} + \sigma_{nj} \cdot \varepsilon_{nj}$ 
7        $\mathcal{L} \leftarrow \mathcal{L} + \frac{1}{2} (1 + \ln \sigma_{nj}^2 - \sigma_{nj}^2 - \mu_{nj}^2)$ 
8     }
9      $\mathcal{L} \leftarrow \mathcal{L} + \ln p(x_n | z_n, w)$ 
10     $w \leftarrow w - \eta * \nabla_w \mathcal{L}$ 
11     $\varphi \leftarrow \varphi - \eta * \nabla_\varphi \mathcal{L}$ 
12  }

```

Generative Adversarial Networks

5.1 Introdução

Esse capítulo trata de um método de aprendizado não supervisionado usado para treinar modelos **generativos**. Esses modelos são capazes de gerar novos exemplos que se assemelham aos dados de treinamento. A ideia central é simples. Queremos que as novas amostras geradas por nosso modelo generativo sejam tão boas que seja difícil afirmar qual é a amostra real e qual é a amostra gerada. Para isso, utilizamos uma abordagem de aprendizado adversarial, onde dois modelos competem entre si: um gerador e um discriminador.

Enquanto o modelo generativo cria imagens, o discriminador tenta distinguir entre imagens reais e imagens geradas. O objetivo do gerador é enganar o discriminador, enquanto o objetivo do discriminador é identificar corretamente as imagens reais e falsas. Esse processo de competição leva a uma melhoria contínua de ambos os modelos, resultando em um gerador capaz de produzir amostras realistas.

5.2 Treinamento Adversarial

Vamos considerar um modelo generativo baseado numa transformação de uma variável latente z para o espaço dos dados x . Por simplicidade, vamos definir

$$p(z) = N(0, I) \quad (112)$$

juntamente de uma transformação não-linear $g(z, w)$ definida por uma rede neural profunda com parâmetros w conhecida como **gerador**. Essas definições implicam que há uma distribuição de probabilidade sobre x que queremos encaixar em cima dos nossos dados. No entanto, nós não conseguimos determinar w maximizando a verossimilhança, já que em geral não tem forma fechada.

Como já explicamos, a ideia das GANs é introduzir uma segunda rede que vai ser treinada em conjunto com a rede **geradora**, a chamada **discriminadora**. Seu trabalho é distinguir entre amostras reais e amostras geradas. O discriminador é definido como uma rede neural profunda $d(x, \varphi)$ com parâmetros φ que retorna a probabilidade de x ser uma amostra real. O discriminador é treinado para maximizar a probabilidade de classificar corretamente as amostras reais e falsas, enquanto o gerador é treinado para fazer com que a probabilidade seja a mais próxima possível de 0.5, ou seja, o gerador quer enganar o discriminador.

5.3 Função de Custo

Para definir isso precisamente, vamos definir uma variável binária:

$$t = \begin{cases} 1 & \text{se } x \text{ é real} \\ 0 & \text{se } x \text{ é sintética} \end{cases} \quad (113)$$

a rede discriminadora possui um único output com uma ativação sigmoideal. Esse output representa a probabilidade de x ser real, ou seja

$$d(x, \varphi) = \mathbb{P}(t = 1 | x, \varphi) \quad (114)$$

nós treinamos a rede discriminadora usando a clássica função cross-entropy

$$E(w, \varphi) = -\frac{1}{N} \sum_{n=1}^N \{t_n \ln d(x_n, \varphi) + (1 - t_n) \ln(1 - d(x_n, \varphi))\} \quad (115)$$

O dataset utilizado tem tanto as amostras reais x_n quanto as sintéticas z_n geradas pelo gerador. Então podemos separar o dataset D em D_{real} e $D_{\text{sintético}}$. Dessa forma, como definimos que $t = 1$ para amostras reais e $t = 0$ para amostras sintéticas, podemos reescrever a função de custo como:

$$E(w, \varphi) = -\frac{1}{N_{\text{real}}} \sum_{n \in D_{\text{real}}} \ln d(x_n, \varphi) - \frac{1}{N_{\text{sintético}}} \sum_{n \in D_{\text{sintético}}} \ln(1 - d(z_n, \varphi)) \quad (116)$$

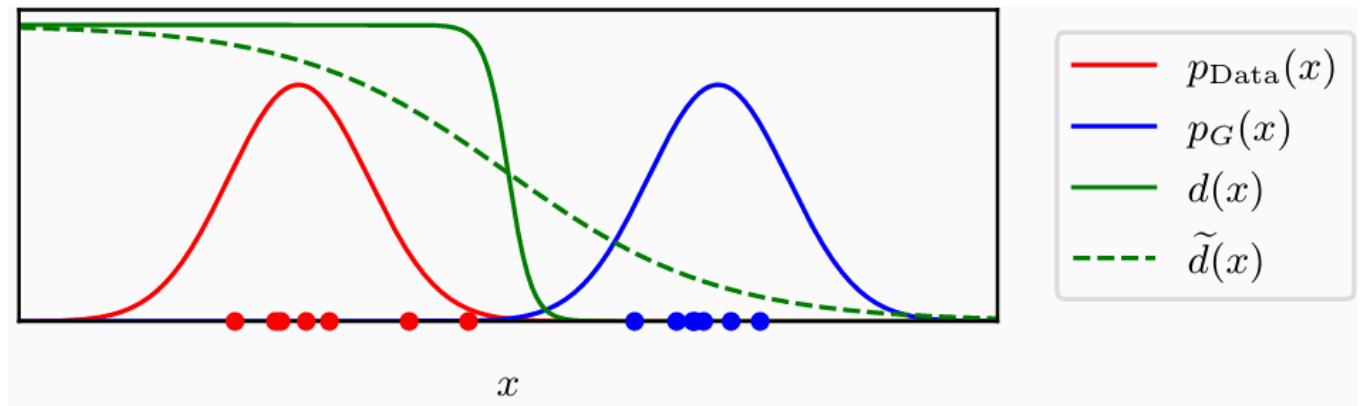
onde (normalmente) $N_{\text{real}} = N_{\text{sintético}}$, ou seja, o dataset é balanceado. O interessante desse método é que podemos otimizar a função com base no gradiente, no entanto, nós a minimizamos com relação a φ e **maximizamos** com relação a w . Ou seja, o gerador quer maximizar a função de custo, enquanto o discriminador quer minimizar. Isso é conhecido como **jogo de soma zero**.

O modo de treino que apresentamos até o momento faz com que o gerador aprenda distribuições não-condicionais $p(x)$. Por exemplo, se ele for treinado com imagens de cachorros, ele vai aprender a gerar imagens de cachorros. No entanto, podemos criar GANs condicionais, onde o gerador aprende uma distribuição $p(x|c)$ onde c pode, por exemplo, representar um vetor que especifica a raça do cachorro.

5.4 Treinamento do GAN

Por mais que essa abordagem de treinamento seja interessante e gere ótimos resultados, ela possui algumas dificuldades por conta do aprendizado adversarial.

Um desafio que pode surgir durante o treinamento é o **colapso do modo** (mode collapse). Isso ocorre quando o gerador aprende a produzir apenas um conjunto limitado de amostras, ignorando a diversidade presente nos dados reais. Como resultado, o gerador pode gerar imagens muito semelhantes entre si, mesmo que os dados reais sejam variados. Por exemplo, num dataset de dígitos, o gerador pode aprender a gerar apenas o dígito “3”, mesmo que o dataset contenha todos os dígitos de 0 a 9. Isso indica que o gerador não está capturando a diversidade dos dados reais, resultando em uma representação limitada do espaço de entrada. Isso ocorre pois o gerador encontra um ponto ótimo local que engana o discriminador, mas não representa a distribuição real dos dados.



Essa imagem mostra um exemplo dos dados reais providos da distribuição **fixa**, mas **desconhecida**, $p_{\text{Data}}(x)$ e os dados do gerador $p_G(x)$. Podemos ver que, como os dados são muito distintos, o discriminador consegue facilmente distinguir entre eles. No entanto, justamente por conta dos dados iniciais do gerador serem tão diferentes e o discriminador classificá-los tão bem, o treinamento do gerador não é eficiente, pois pequenas alterações no seu processo de geração de amostras não vão enganar o discriminador. Para contornar isso, podemos utilizar de uma função discriminadora mais suave, na imagem representada por $\tilde{d}(x)$

Referências

[1] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.